MIT 8103 Advanced Database Systems: Portfolio Summary

Name: Ogboye Michael

Student ID: 301931356

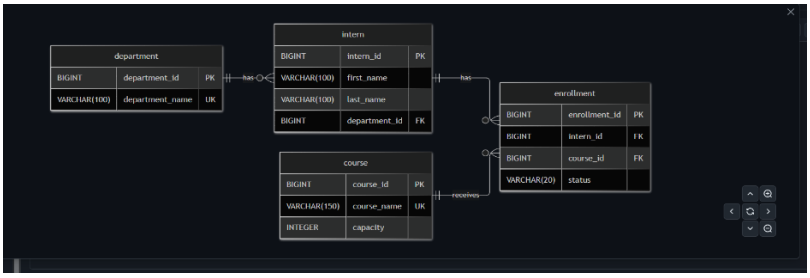
Repository: https://github.com/mogboye1356/Ogboye_Michael_301931356.git

Case study

Upperlink LaunchPad is an Intern Learning Management System (LMS) designed to track intern details, departments, courses, and enrollments. It provides a structured database backbone for managing training tracks, onboarding flows, and milestone events for interns.

Portfolio 1: Database Design and Modelling

I designed a relational PostgreSQL database to model the LMS. The schema includes core tables for departments, interns, courses, and enrollments. I applied normalization to reduce data redundancy and set primary and foreign keys to enforce strict referential integrity.



ER Diagram

Portfolio 2: Query Processing and Optimization

I analyzed a read-heavy query using EXPLAIN ANALYZE. Initially, the database used a sequential scan to find records. To speed this up, I created a B-tree index on the filtered column.

Metric	Before Indexing	After Indexing
Scan Type	Sequential Scan	Index Scan
Execution Time	5.880 ms	0.050ms

Meaning: Adding the index allowed PostgreSQL to use a lookup tree instead of scanning every single row, which dropped the execution time from 5.880 ms to 0.050 ms. This kind of optimization is necessary to keep queries fast as the LMS historical records grow.

Portfolio 3: Transactions and Concurrency

I tested PostgreSQL's concurrency controls by running a script with two simultaneous sessions (Session A and Session B). This simulated two administrators trying to update conflicting enrollment records at the same time.

Result: Session B was forced to wait for Session A to commit its transaction due to row level locking. Once Session A committed, Session B received a duplicate key constraint error.

Meaning: The database's ACID properties (specifically Isolation and Consistency) prevented a race condition, ensuring the system didn't create duplicate active enrollments.

Portfolio 4: NoSQL and Advanced Data Models

I set up a local MongoDB database (launchpad_events) for flexible event logging to complement the strict PostgreSQL schema. I inserted four JSON documents representing "enrolled" and "completed" milestones.

Result: A query filtering for details.final_grade: "A" successfully returned exactly one document detailing Ada James completing the UI/UX Principles course.

Meaning: The NoSQL document model lets me store different event details (like an initial status versus a final grade) in the same collection without leaving empty or null fields. This keeps the event timeline scalable and separate from the core relational database.

Portfolio 5: Distributed and Cloud Database Exercise

Note: This section discusses a proposed design; a working replica set was not implemented or tested in the Codespace.

I outlined a theoretical distributed architecture for the MongoDB event records to ensure high availability. The design uses a 3-node replica set spread across different failure zones (one Primary, two Secondaries).

Meaning: If the primary node fails, the remaining nodes can elect a secondary as the new primary to minimize downtime. I also proposed using majority write concern to prevent data loss. Sharding wasn't necessary given the small volume of events.

Evidence

All SQL files, scripts, and screenshot evidence are maintained in the GitHub repository:

- Portfolio 1
- Portfolio 2
- Portfolio 3
- Portfolio 4
- Portfolio 5

Personal reflection

When designing the database, I chose to keep intern and enrollment records separate. I record the intern details once, and use the enrollment record to track when that intern takes a specific course (such as React Basics). My design assumption was that interns remain in a single department for the duration of their placement, so I stored department_id directly in the intern table rather than creating a separate placement history table.

The biggest difficulty I faced was fixing the MongoDB Codespace environment and troubleshooting installation errors. I solved this by using AI to break down the installation steps and understand how to navigate the Ubuntu terminal. From this, I learned the importance of carefully following terminal instructions and ensuring I am operating in the correct environment specifically learning to distinguish between the test> shell and the Linux $ prompt, and using the correct exit commands like \q or exit.

References

- PostgreSQL Global Development Group. (n.d.). *PostgreSQL Documentation*.
- MongoDB, Inc. (n.d.). *MongoDB Documentation: Insert Documents*.
- MongoDB, Inc. (n.d.). *MongoDB Documentation: Replication*.

AI-use declaration

I used Gemini AI to assist with proofreading for grammatical clarity, troubleshoot terminal environment setups, and clarify database concepts. I personally ran the database commands within the GitHub Codespace, compared the results against the evaluation criteria, and corrected any issues prior to submission. I fully understand the work and remain responsible for its accuracy.